CPSY 1291 · Recitation handout 11

Attention and the transformer

What Assignment 4 needs from Lecture 17

Fall 2026 · Week 11 · hold this after Lecture 17; Assignment 4 is due Tue 11/24

What this session is for

This is the last handout about content. Assignment 4 covers Lectures 15–17 and is due on Tuesday 11/24. Recitation 10 did recurrence and dynamics (Lectures 15 and 16); this handout finishes the assignment's scope with Lecture 17. Generative models (Lecture 14) and large language models (Lecture 18) are not on the assignment; they are on Exam 2, and Recitation 12 reviews them.

Read it in two passes. Section 1 (attention) is the part everything else depends on, and it is worth doing slowly with a pencil. Sections 2 and 3 are short.

> **The one idea.** Attention computes a set of *weights* that say how much each position should contribute, and takes a weighted average of what the positions offer. A modern Hopfield update does exactly the same thing, with the cue as the query and the stored patterns as keys and values (Section 3). The differences are in what is averaged and what determines the weights.

1 Attention

1.1 Tokens are a matrix

A sequence of T tokens, each embedded as a d-dimensional vector, is a matrix $\boldsymbol{X}$ of shape (T, d). Everything in a transformer operates on that matrix. Note what has been given up compared to an RNN: there is no order in a matrix's rows unless you put it there, which is why a learned table of **positional embeddings** is added to each symbol's vector. What is gained: any position reaches any other in one step, and all T rows come out of a few matrix products. The price is $T \times T$ scores, so doubling the sequence length quadruples the cost.

1.2 Q, K, V are three views of the same input

```
Q = X @ Wq # (T, d_k) what each position is looking for
K = X @ Wk # (T, d_k) what each position offers
V = X @ Wv # (T, d_v) what each position would contribute
```

All three are linear projections of the *same* $\boldsymbol{X}$ — that is what makes it *self*-attention. The names are a retrieval metaphor: a query is matched against keys, and the matching keys' values are returned.

1.3 Scaled dot-product attention

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}$$

Masked positions get a score of $-\infty$ before the softmax, so their weight is exactly 0. Writing this as code is part of Assignment 4.

Track the shapes, because they are the whole thing:

Step	Shape	Meaning
$\boldsymbol{Q}\boldsymbol{K}^\top$	(T, T)	every position against every position
after softmax	(T, T)	each *row* is a distribution over positions
$\boldsymbol{A}\boldsymbol{V}$	(T, d_v)	each position: a weighted average of all values

> **Three details that are not decoration.** **The $\sqrt{d_k}$**: dot products of d_k-dimensional vectors grow like $\sqrt{d_k}$, and a softmax of large numbers saturates to a one-hot vector with almost no gradient. Remove it and training fails for large d_k. **`dim=-1`**: the softmax runs across *keys*, so each row is a distribution over what to attend to. Softmax over the wrong axis produces nonsense that runs perfectly. **The mask is $-\infty$, not 0**: it must be applied *before* the softmax, because zeroing afterwards leaves rows that no longer sum to one.

1.4 Multi-head: split, attend, concatenate

Split the d dimensions into H groups, run the same operation on each, concatenate the results, and let a final matrix $\boldsymbol{W}_O$ mix the heads back to width d. Each head has its own three projections, so different heads can attend to different things at the cost of one wide head.

```
# (N, T, d) -> (N, H, T, d//H)
Q = Q.view(N, T, H, d // H).transpose(1, 2)
# ... attention exactly as above, batched over the H axis ...
out = out.transpose(1, 2).reshape(N, T, d) # concatenate the heads back
```

The `transpose(1, 2)` before and after is the only subtle part. It puts the head axis next to the batch axis so that the matrix multiplies treat heads as independent problems.

1.5 Reading an attention map, carefully

The (T, T) matrix $\boldsymbol{A}$ is tempting to plot and interpret. Single heads in trained language models do track relations people can name — verb to object, pronoun to referent — but three cautions apply:

- There are H heads per layer and many layers, so "the attention map" is one choice among many.

- A map shows *where* a head reads, not *why* the output is what it is.
- A map is a hypothesis. Changing the input, or switching the head off, tests it.

> **The one-sentence version.** An attention map is a hypothesis about where information is read, and needs an intervention before it explains anything — the discipline of the attribution maps in Recitation 9.

2 Training and generation

2.1 One forward pass, T training examples

A language model predicts the next symbol at every position. The target at position t is the symbol at $t+1$, and the loss is the cross-entropy averaged over positions, in nats:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log \hat{y}^{\langle t \rangle}_{y^{\langle t+1 \rangle}}.$$

The **causal mask** sets every score above the diagonal to $-\infty$, so position t cannot read the symbol it must predict. With the mask, one forward pass gives all T predictions. The input is the true text, not the model's own guesses: this is teacher forcing, from Recitation 10.

2.2 The bigram baseline

Count, in the training text, how often each symbol follows each other one:

$$P(\text{next} = b \mid \text{current} = a) = \frac{\text{count}(a \to b)}{\text{count}(a)}.$$

Score the table with the same cross-entropy, on the same held-out text. A bigram table reads one symbol of context and the transformer can read the whole left context, so the gap between the two losses is what the longer context is worth. As in the baselines lecture, a score means something only next to a baseline computed on the same data.

2.3 Generation is a loop

```
for _ in range(n_new):
   logits = model(idx)[:, -1, :] / temperature # last position only
   probs = torch.softmax(logits, dim=-1)
   nxt = torch.multinomial(probs, num_samples=1) # SAMPLE, do not argmax
   idx = torch.cat([idx, nxt], dim=1)
```

If the text grows longer than the context, keep only the last T symbols.

`temperature`	What it does
$\to 0$	approaches greedy: repetitive, often loops
$= 1$	samples from the model's own distribution
> 1	flatter, more surprising, less coherent

Sampling is not a detail of the model. Two very different outputs from the same weights can differ only in the temperature, so any claim about "what the model does" has to state it.

3 The Hopfield update is one attention step

A modern Hopfield network keeps its stored patterns as the rows of $\boldsymbol{X}$ and recalls from a cue $\boldsymbol{s}$ with a softmax:

$$\boldsymbol{s} \;\leftarrow\; \boldsymbol{X}^\top \operatorname{softmax}\big(\beta\, \boldsymbol{X}\, \boldsymbol{s}\big).$$

Read it in three steps: $\boldsymbol{X}\boldsymbol{s}$ gives one match score per stored pattern, the softmax turns the scores into weights, and $\boldsymbol{X}^\top$ times the weights is a weighted average of the stored patterns. That is attention for one query, $\operatorname{softmax}(\boldsymbol{q}\boldsymbol{K}^\top/\sqrt{d_k})\boldsymbol{V}$, with the cue as the query, the stored patterns as both keys and values (no learned projections: $\boldsymbol{W}_K = \boldsymbol{W}_V = \boldsymbol{I}$), and β in the role of $1/\sqrt{d_k}$. A large β sharpens the weights, so recall picks out one stored pattern.

4 Exercises

1. You delete the $\sqrt{d_k}$ from scaled dot-product attention. Training works at $d_k = 8$ and fails at $d_k = 512$. Why?

2. You apply a causal mask by multiplying the attention weights by 0 *after* the softmax. What is wrong?

3. You take softmax over `dim=-2` instead of `dim=-1`. Does anything error? What does the model now compute?

4. A colleague shows an attention map and says it proves the model bases its answer on the word "not". Give two objections, and the test that would settle it.

5. Generation at `temperature=0.1` produces the same sentence repeated forever. Explain in terms of the sampling loop.

6. In the toy text "that hat ate the tea", what is $P(\text{next} = \text{a} \mid \text{current} = \text{h})$ under a bigram table? Your transformer's held-out loss equals the table's. What has it learned?

7. In the modern Hopfield update, which quantity plays the query, which the keys, which the values, and which $1/\sqrt{d_k}$?

Answers

1. Dot products of d_k-dimensional vectors have magnitude growing like $\sqrt{d_k}$. At $d_k = 512$ the unscaled scores are large, the softmax saturates to nearly one-hot, and its gradient is nearly zero, so nothing trains. At $d_k = 8$ the scores are small enough that it hardly matters.

2. The softmax has already distributed weight to the masked positions, so zeroing afterwards leaves rows that do not sum to one — and the unmasked positions get less than their share. Mask with $-\infty$ *before* the softmax.

3. Nothing errors: the matrix is (T, T), so both axes are valid. You now normalize down the columns, so each row is no longer a distribution over positions and the output is a weighted combination with weights that do not sum to one. It trains, badly, with no diagnostic.

4. (i) There are many heads and many layers; "the attention map" is one choice among many. (ii) A map shows where a head reads, not why the output is what it is. The test is an intervention: change the input (remove or replace "not") or switch the head off, and see whether the answer changes.

5. At low temperature the softmax is nearly one-hot, so sampling is effectively `argmax`. The model picks its most likely continuation every time, and always taking the most likely symbol tends to loop. Raising the temperature breaks the loop.

6. The symbol h is followed by a twice and by e once, so $2/3$. A transformer that only matches the bigram table has used no more than one symbol of context: its attention is not reading the rest of the left context. Check the mask, the positional embeddings, and whether it trained at all.

7. The cue $\boldsymbol{s}$ is the query; the stored patterns $\boldsymbol{X}$ are both the keys and the values; β plays $1/\sqrt{d_k}$.

Cheat sheet

You want	You write
attention scores	`Q @ K.transpose(-2,-1) / sqrt(d_k)`
a causal mask	`masked_fill(mask == 0, float('-inf'))` *before* softmax
the softmax axis	`dim=-1` — over keys, always
split into heads	`view(N,T,H,d//H).transpose(1,2)`
the next-symbol target	the input shifted by one position
a bigram baseline	`count(a→b) / count(a)`, scored on held-out text
sample a token	`torch.multinomial(probs, 1)` — not `argmax`
a modern Hopfield update	`X.T @ softmax(beta * X @ s)`